

Persistence



IS2205 - Mobile Application Design & Development

Storing Data



Ways to store data in an Android app

- App-specific storage (files to be private to the app)
- Shared storage (files to be shared with other apps)
- Preferences (private simple data)
- Databases (structured data – private)

When to use which?

- **App-specific storage:**

Data is only used by the respective app

Data is meaningful to the app only

Relatively limited amounts of data

When to use which?

- **Shared storage**

Data may be used by other apps/services

Data is less sensitive or encryption is possible

When to use which?

- **Preferences**

Data is private

Structure is present

Structure is less complex

When to use which?

- **Databases**

Data is structured

Data has complex relationships

Relatively large size

Note: in-built RDBMS in Android API is private

App-specific storage

- Files meant for your app's use only
- From **internal storage** or from **external storage**
- Retention app installed time only
- `getFilesDir()` / `getCacheDir()` : **internal**
- `getExternalFilesDir()` / `getExternalCacheDir()` :
external

App-specific storage...(internal)

- From internal storage:

- context.filesDir

```
val file = File(context.filesDir, filename)
```

- Outputting to a file

```
val filename = "myfile"
val fileContents = "Hello world!"
context.openFileOutput(filename, Context.MODE_PRIVATE).use {
    it.write(fileContents.toByteArray())
}
```

App-specific storage...(cache)

- From internal storage:
 - context.cacheDir

```
File.createTempFile(filename, null, context.cacheDir)
```

- Files in cache directory

```
val cacheFile = File(context.cacheDir, filename)
```

App-specific storage...(External)

- External storage is larger compared to limited internal storage:
- App is given **files** directory and **cache** directory in external storage as well.
- Above files are app specific and no added permissions needed.

```
// Checks if a volume containing external storage is available to at least read.  
fun isExternalStorageReadable(): Boolean {  
    return Environment.getExternalStorageState() in  
        setOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)  
}
```

App-specific storage...(External)

- If there are multiple volumes identified as external:

```
val externalStorageVolumes: Array<out File> =  
    ContextCompat.getExternalFilesDirs(applicationContext, null)  
val primaryExternalStorage = externalStorageVolumes[0]
```

- Cache directory (external):

```
val externalCacheFile = File(context.externalCacheDir, filename)
```

Shared storage

- With other apps (meant to be so)
- Good for shareable media files (image, audio, video)
- Need additional permissions `READ_EXTERNAL_STORAGE`
`WRITE_EXTERNAL_STORAGE` (Before Android 13)
- Files retain even without the app

Shared storage : Different types

- Android has standard public directories for each photos, audio and video. Apps can access them via MediaStore API.
- Another public directory for documents such as PDF, EPUB etc. Apps can access them via Storage Access Framework.
- Datasets: On Android ≥ 11 the system caches large datasets that multiple apps might use. Apps can access these via the BlobStoreManager API.

Shared Preferences : Exercise

There is another form of Android storage called shared preferences (SP). Write a note on the main features of the this mode considering: (a)the permissions, (b)what parameters needed to store via SP, (c)What are some important methods used for this API? What type of use-cases are good for SP?

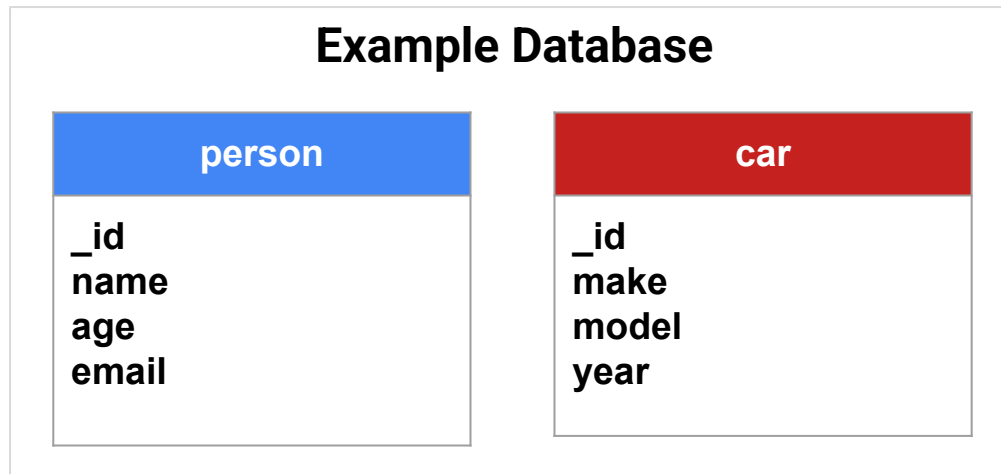
This is an individual activity. (Also include your resource you referred)

Quiz : VLE

What is a database?

Collection of structured data that can be easily accessed, searched, and organized, consisting of:

- Tables
- Rows
- Columns

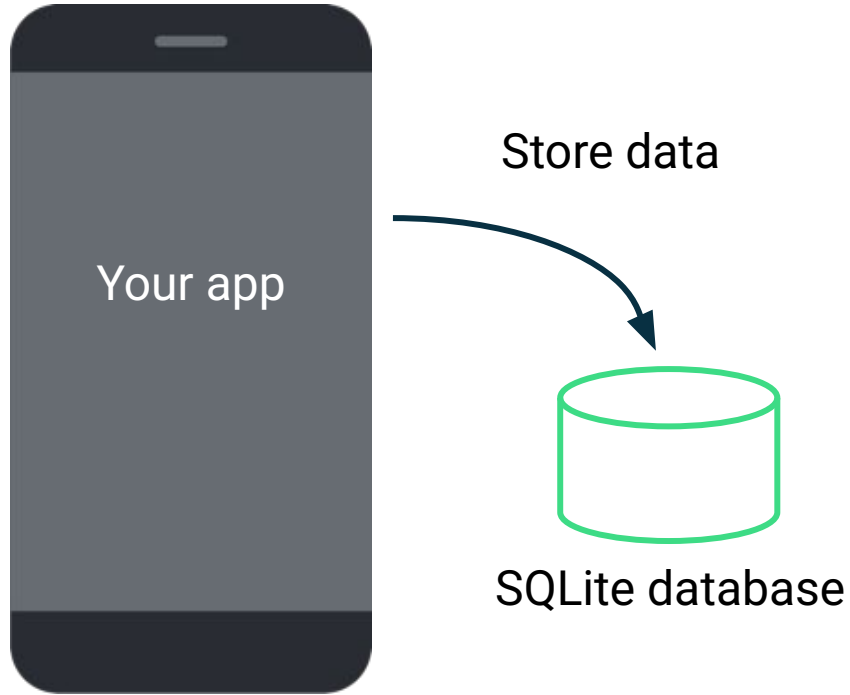


Structured Query Language (SQL)

Use SQL to access and modify a relational database.

- Create new tables
- Query for data
- Insert new data
- Update data
- Delete data

SQLite in Android



Example SQLite commands

Create `INSERT INTO colors VALUES ('red', '#FF0000');`

Read `SELECT * from colors;`

Update `UPDATE colors SET hex='#DD0000' WHERE name='red';`

Delete `DELETE FROM colors WHERE name = 'red';`

Interacting with a database

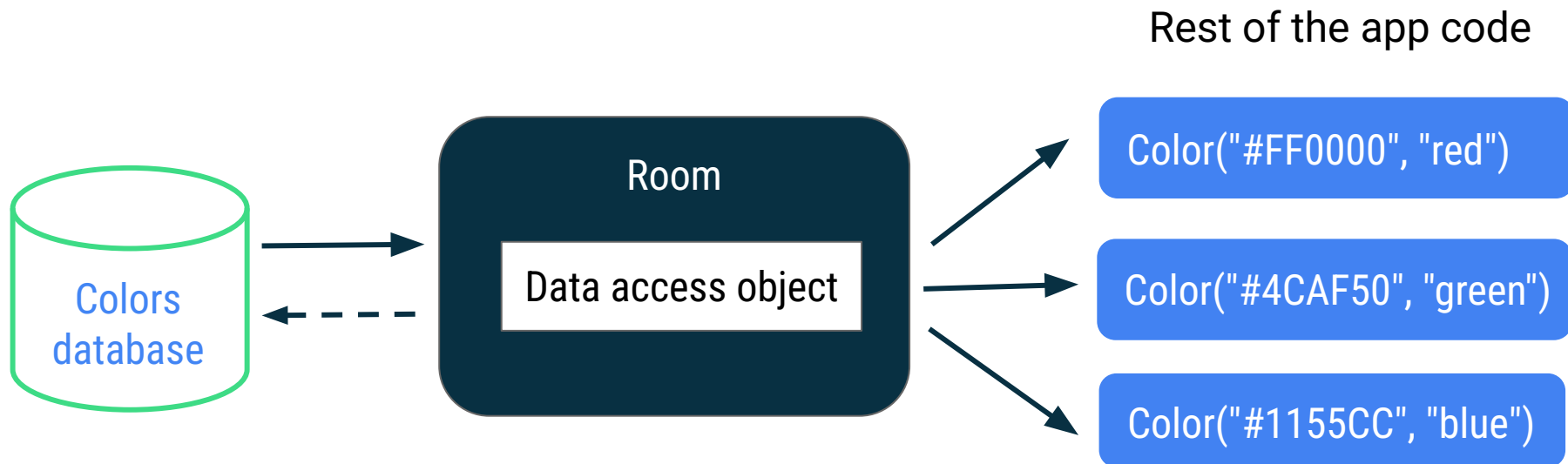
- Direct SQLite interaction : Historical
- No compile-time verification of raw SQL queries
- Need lots of boilerplate code to convert between SQL queries ↔ data objects

Room persistence library

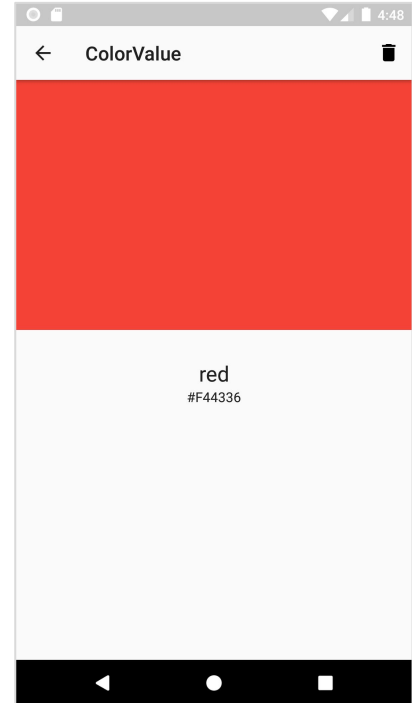
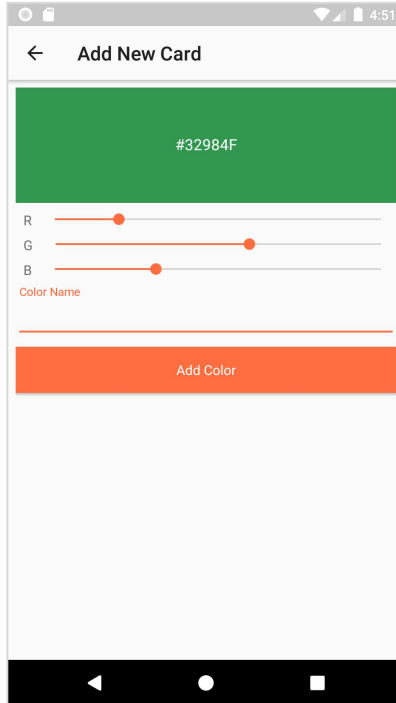
Add Gradle dependencies

```
dependencies {  
    val room_version = "2.X.X"  
    implementation("androidx.room:room-runtime:$room_version")  
  
    // Kotlin Symbol Processor for annotation processing  
    ksp("androidx.room:room-compiler:$room_version")  
  
    // Kotlin Extensions and Coroutines support for Room  
    implementation "androidx.room:room-ktx:$room_version"  
}
```

Room



ColorValue app



Room

- Entity `Color`
- DAO `ColorDao`
- Database `ColorDatabase`

Color class : POJO replacement

```
data class Color(  
    @PrimaryKey(autoGenerate = true)  
    val _id: Int = 0  
    val hex: String  
    val name: String  
)
```

Annotations

- Provide extra information to the compiler

`@Entity` marks entity class, `@Dao` for DAO, `@Database` for database

- Can take parameters

```
@Entity(tableName = "colors")
```

- Can autogenerate code for you

Entity

Class that maps to a SQLite database table

- `@Entity`
- `@PrimaryKey`
- `@ColumnInfo`

Example entity

```
@Entity(tableName = "colors")
```

```
data class Color (
    @PrimaryKey(autoGenerate = true) val _id: Int,
    @ColumnInfo(name = "hex_color") val hex: String,
    val name: String
)
```

colors

_id
hex_color
name

Data access object (DAO)

Work with DAO classes instead of accessing database directly:

- Define database interactions in the DAO.
- Declare DAO as an interface or abstract class.
- Room creates DAO implementation at compile time.
- Room verifies all of your DAO queries at compile-time.

Example DAO

@Dao

```
interface ColorDao {  
    @Query("SELECT * FROM colors")  
    fun getAll(): Array<Color>  
    @Insert  
    fun insert(vararg color: Color)  
    @Update  
    fun update(color: Color)  
    @Delete  
    fun delete(color: Color)
```


Query

@Dao

```
interface ColorDao {  
    @Query("SELECT * FROM colors")  
    fun getAll(): Array<Color>  
  
    @Query("SELECT * FROM colors WHERE name = :name")  
    fun getColorByName(name: String): LiveData<Color>  
  
    @Query("SELECT * FROM colors WHERE hex_color = :hex")  
    fun getColorByHex(hex: String): LiveData<Color>
```

Insert

@Dao

```
interface ColorDao {
```

```
    ...
```

@Insert

```
    fun insert(vararg color: Color)
```

```
    ...
```

```
}
```

Update

@Dao

```
interface ColorDao {
```

```
    ...
```

@Update

```
    fun update(color: Color)
```

```
    ...
```

```
}
```

Delete

@Dao

```
interface ColorDao {
```

```
    ...
```

@Delete

```
    fun delete(color: Color)
```

```
    ...
```

```
}
```

Create a Room database

- Annotate class with `@Database` and include list of entities:
`@Database(entities = [Color::class], version = 1)`
- Declare abstract class that extends `RoomDatabase`:
`abstract class ColorDatabase : RoomDatabase() {`
 - Declare abstract method with no args that returns the DAO:
`abstract fun colorDao(): ColorDao`

Example Room database

```
@Database(entities = [Color::class], version = 1)
abstract class ColorDatabase : RoomDatabase() {
    abstract fun colorDao(): ColorDao
    companion object {
        @Volatile
        private var INSTANCE: ColorDatabase? = null
        fun getInstance(context: Context): ColorDatabase {
            ...
        }
    }
}
```

Create database instance

```
fun getInstance(context: Context): ColorDatabase {  
    return INSTANCE ?: synchronized(this) {  
        INSTANCE ?: Room.databaseBuilder(context.applicationContext,  
            ColorDatabase::class.java, "color_database")  
                .fallbackToDestructiveMigration()  
                .build()  
                .also { INSTANCE = it }  
    }  
}
```

Get and use a DAO

Get the DAO from the database:

```
val colorDao = ColorDatabase.getInstance(application).colorDao()
```

Create new Color and use DAO to insert it into database:

```
val newColor = Color(hex = "#6200EE", name = "purple")
```

```
colorDao.insert(newColor)
```


Last slide for the IS Students

Thank You!